

Experiment:21

Develop a C program to implement worst fit algorithm of memory management.

Code: -

```
#include <stdio.h>

#define MAX_MEMORY 1000

int memory[MAX_MEMORY];

void initializeMemory()
{
    for (int i = 0; i < MAX_MEMORY; i++)
    {
        memory[i] = -1;
    }
}

void displayMemory()
{
    int i, j;
    int count = 0;
    printf("\nMemory Status:\n");
    for (i = 0; i < MAX_MEMORY; i++)
    {
        if (memory[i] == -1)
        {
            count++;
            j = i;

            while (j < MAX_MEMORY && memory[j] == -1)
```

```

        {
            j++;
        }
printf("Free memory block %d-%d\n", i, j - 1);
i = j - 1;
    }
}
if (count == 0)
{
    printf("No free memory available.\n");
}
}
void allocateMemory(int processId, int size)
{
    int worstStart = -1;
    int worstSize = 0;
    int start = -1;
    int blockSize = 0;
    for (int i = 0; i <= MAX_MEMORY; i++)
    {
        if (i < MAX_MEMORY && memory[i] == -1)
        {
            if (blockSize == 0)
            {
                start = i;
            }

```

```
        blockSize++;
    }
    else
    {
        if (blockSize >= size && blockSize > worstSize)
        {
            worstSize = blockSize;
            worstStart = start;
        }

        blockSize = 0;
    }
}

if (worstStart != -1)
{
    for (int i = worstStart; i < worstStart + size; i++)
    {
        memory[i] = processId;
    }

    printf("\nAllocated memory block %d-%d to Process %d\n",
        worstStart,
        worstStart + size - 1,
        processId);
```

```

    }
    else
    {
        printf("\nMemory allocation for Process %d failed "
              "(not enough contiguous memory).\n", processId);
    }
}

void deallocateMemory(int processId)
{
    for (int i = 0; i < MAX_MEMORY; i++)
    {
        if (memory[i] == processId)
        {
            memory[i] = -1;
        }
    }
    printf("\nMemory released by Process %d\n", processId);
}

int main()
{
    initializeMemory();
    displayMemory();

    allocateMemory(1, 200);
    displayMemory();
    allocateMemory(2, 300);

```

```
displayMemory();  
deallocateMemory(1);  
displayMemory();  
allocateMemory(3, 400);  
displayMemory();  
return 0;  
}
```

Output:

```
Memory Status:  
Free memory block 0-999  
  
Allocated memory block 0-199 to Process 1  
  
Memory Status:  
Free memory block 200-999  
  
Allocated memory block 200-499 to Process 2  
  
Memory Status:  
Free memory block 500-999  
  
Memory released by Process 1  
  
Memory Status:  
Free memory block 0-199  
Free memory block 500-999  
  
Allocated memory block 500-899 to Process 3  
  
Memory Status:  
Free memory block 0-199  
Free memory block 900-999  
  
...Program finished with exit code 0  
Press ENTER to exit console.□
```